SAVE THIS POST

THE ULTIMATE

# C#

# CHEAT SHEET

OOP | Keywords | Collections | Async/Await | SOLID

Delegates | Exceptions | Generics

32 SLIDES | CONCEPTS + CODE EXAMPLES

*Swipe to start >>*

.NET Developer

↻ Repost

Follow for more →

# C# BASICS & DATA TYPES

## VALUE TYPES

*Stores value directly in memory*

| | |
|---|---|
| `int, short, long` | Integer numbers |
| `float, double, decimal` | Real numbers |
| `char` | Single character |
| `bool` | True / False |
| `struct` | Custom value type |
| `enum` | Named constants |

## REFERENCE TYPES

*Stores address (pointer) to data*

| | |
|---|---|
| `string` | Text (immutable) |
| `array` | Fixed-size collection |
| `class` | Blueprint for objects |
| `object` | Base type of all |
| `delegate` | Function pointer |
| `interface` | Contract / API |

## SPECIAL TYPES

| | |
|---|---|
| `var (Implicit)` | Compiler decides type at compile time |
| `dynamic` | Type resolved at runtime |
| `Nullable (int?)` | Value types that can also hold null |
| `Generics <T>` | Type-safe reusable code |

*Boxing (value to object) and Unboxing (object to value) hurt performance!*

↻ Repost

.NET Developer

Follow for more →

# C# BASICS CODE EXAMPLES

### Constant vs ReadOnly

```
const int MaxAge = 100;
// Value fixed at compile time

readonly int StartYear;
StartYear = DateTime.Now.Year;
// Set once at runtime
```

### Value Type vs Reference Type

```
int a = 10;  int b = a;
b = 20;
// a = 10, b = 20 (separate copies)

int[] arr1 = {1,2,3};
int[] arr2 = arr1;
// Both point to same memory
```

### Nullable Type

```
int? age = null;
if (age.HasValue)
  Console.Write(age.Value);
else
  Console.Write("No age");
```

### Implicit Type (var)

```
var name = "Raju";
// Compiler knows it's string

var num = 100;
// Compiler knows it's int
```

### Generics Example

```
class Box<T>
{
  public T Value { get; set; }
}

Box<int> b = new Box<int>();
b.Value = 42;
```

### Type Conversion

```
// Implicit (safe, auto)
int num = 10;
double d = num;

// Explicit (manual cast)
double x = 9.8;
int y = (int)x; // y = 9
```

### Boxing & Unboxing

```
// Boxing: value -> object
int num = 42;
object obj = num;

// Unboxing: object -> value
int val = (int)obj;
```

### Anonymous Type

```
var student = new
{
  Name = "Ravi",
  Age = 20
};
// Compiler creates the type
```

### Enum Example

```
enum Day
{ Mon, Tue, Wed, Thu, Fri }

Day today = Day.Mon;
Console.Write(today); // Mon
```

### String vs StringBuilder

```
string s = "Hello";
s = s + " World"; // New obj

var sb = new StringBuilder();
sb.Append("Hello"); // Same obj
```

# C# BASICS DETAILED ANSWERS

### Q: What is C#?

C# is an object-oriented programming language created by Microsoft. It is Simple, Object Oriented, Type Safe, and supports Structured Programming.

### Q: Why is C# Type Safe?

Type safety in C# prevents an object from sneaking into another object's memory. The compiler enforces strict type checking, ensuring variables only hold values of their declared type.

### Q: Constant vs ReadOnly?

**Constant:** For absolute constants value does not change during program execution.
**ReadOnly:** For runtime constants initialized at runtime, then treated as constant.

### Q: Value Type vs Reference Type?

**Value Type:** Stores value directly on stack at compile time. Removed when out of scope.
**Reference Type:** Stores address on heap at runtime. Garbage collected by CLR.
Examples Value: int, float, bool, struct, Reference: string, array, class, enum

### Q: Special Types Explained?

**Nullable:** Allows assigning null to value type variables.
**Implicit (var):** No need to define data type compiler infers it at compile time.
**Anonymous:** Creates new types without defining them; compiler decides the type.
**Generics:** Define classes/methods with placeholder T. Advantages: reusability, type safety, performance.

# OOP: ENCAPSULATION & INHERITANCE

## ENCAPSULATION

Wrap data + methods in a single unit

**How to implement:**
1. Declare fields as private
2. Use Properties (get/set)
3. Control access from outside

*Like a capsule: medicine hidden inside*

## INHERITANCE

| | |
|---|---|
| Single | A : B |
| Multilevel | A : B : C |
| Hierarchical | B, C : A |
| Multiple | Via interfaces only |

## ACCESS MODIFIERS

| | |
|---|---|
| public | Everywhere |
| private | Same class only |
| protected | Class + derived classes |
| internal | Same assembly |
| protected internal | Same assembly OR derived |

↻ Repost

.NET Developer

Follow for more →

# OOP: ENCAPSULATION & INHERITANCE CODE

### Encapsulation (private + property)

```
class Student
{
  private string _name;

  public string Name
  {
    get { return _name; }
    set { _name = value; }
  }
}
```

### Single Inheritance

```
class Animal
{
  public void Eat()
  { Console.Write("Eating"); }
}

class Dog : Animal
{ }

Dog d = new Dog();
d.Eat(); // Eating
```

### Access Modifiers

```
class Employee
{
  public string Name;  // everywhere
  private int Salary; // this class
  protected int Age;  // + child
  internal string Dept; // assembly
}
```

### Upcasting & Downcasting

```
class Animal { }
class Dog : Animal { }

// Upcasting (child to parent)
Animal a = new Dog();

// Downcasting (parent to child)
Dog d = (Dog)a;
```

### Composition (HAS-A)

```
class Engine
{ public void Start() { } }

class Car
{
  private Engine e = new Engine();
} // Car HAS-A Engine
```

### Multilevel Inheritance

```
class Animal { }
class Dog : Animal { }
class Puppy : Dog { }

// Puppy inherits Dog,
// Dog inherits Animal
// Puppy gets all features
```

### Hierarchical Inheritance (multiple classes, one parent)

```
class Shape { public void Draw() { } }
class Circle : Shape { }   // Circle gets Draw()
class Square : Shape { }   // Square gets Draw()
```

# OOP: ENCAPSULATION & INHERITANCE

## Q: What is Encapsulation?

Wrapping code and data into a single unit. It secures class members from being accessed by other classes.
**Practice:** Declare all variables as private, use C# Properties (getters/setters) to access them.
**Real-Time Example:** A capsule all medicines (variables/methods) are encapsulated inside.

## Q: What is Inheritance (IS-A)?

A process where one object acquires all properties and behaviors of its parent class. Benefits: Code Reuse and Polymorphic behavior.
**Single:** A inherits B , **Multilevel:** A B C
**Multiple:** Via interfaces only , **Hierarchical:** Multiple classes inherit one base
**Composition (HAS-A):** One class contains another class.

## Q: Boxing & Unboxing?

**Boxing:** Value type object reference.
**Unboxing:** Object value type.
**Consequence:** Severely affects performance.

## Q: Upcasting vs Downcasting?

**Upcasting:** Derived class base class.
**Downcasting:** Base class derived class.
Both allow any value to be treated as object.

## Q: What are the 5 Access Modifiers?

`public` Accessible everywhere , `private` Within the class only
`protected` Within class + derived classes
`internal` Same assembly , `protected internal` Same assembly or derived

# OOP: POLYMORPHISM & ABSTRACTION

## POLYMORPHISM = Many Forms

### Compile-Time

Method Overloading
Same name, different params
`Add(int) vs Add(int, int)`

### Run-Time

Method Overriding
virtual + override keywords
`Parent defines, child overrides`

## ABSTRACT CLASS

- Cannot be instantiated
- Can have constructors
- Can have implemented methods
- Can have abstract methods
- Single inheritance only
- Common functionality base

## INTERFACE

- Defines a contract only
- No constructors
- All methods are abstract
- Supports multiple inheritance
- Enables loose coupling
- Great for Dependency Injection

*Abstract class for shared logic, Interface for contracts and DI.*

.NET Developer

↻ Repost

Follow for more →

# OOP: POLYMORPHISM & ABSTRACTION CODE

### Method Overloading (Compile-time)

```
class Math
{
  int Add(int a, int b)
    { return a + b; }
  double Add(double a, double b)
    { return a + b; }
}
// Same name, different params
```

### Method Overriding (Runtime)

```
class Animal
{ public virtual void Speak()
  { Console.Write("..."); } }

class Dog : Animal
{ public override void Speak()
  { Console.Write("Woof"); } }
// virtual in parent, override in child
```

### Abstract Class

```
abstract class Shape
{
  public abstract double Area();
  public void Show()
  { Console.Write(Area()); }
}

class Circle : Shape
{ public override double Area()
    { return 3.14 * 5 * 5; } }
```

### Interface

```
interface IMovable
{
  void Move();
}

class Car : IMovable
{
  public void Move()
  { Console.Write("Driving"); }
}
```

### Multiple Interfaces (C# allows this, not multiple class inheritance)

```
interface IFly { void Fly(); }
interface ISwim { void Swim(); }

class Duck : IFly, ISwim
{ public void Fly() { } public void Swim() { } }
```

### Sealed Class (no inherit)

```
sealed class Config
{
  public string Key = "abc";
} // Can't inherit from this
```

### Operator Overloading

```
class Point
{ public int X, Y;
  public static Point operator+ (Point a, Point b)
  { return new Point { X=a.X+b.X }; } }
```

# OOP: POLYMORPHISM & ABSTRACTION

## Q: What is Polymorphism?

The ability to handle multiple forms of an object. Inheritance inherits fields/methods; polymorphism uses those methods to perform different tasks.

**Compile-Time:** Achieved via Method Overloading (same name, different signatures).

**Run-Time:** Achieved via Method Overriding (override + virtual keywords + inheritance).

## Q: What is Abstraction?

Hiding implementation details and displaying only essential features. Implemented using abstract classes in C#.

**Real-Time Example:** A laptop you switch it on and use it without knowing how circuits work internally.

**vs Encapsulation:** Abstraction = design level, Encapsulation = implementation level.

## Q: Abstract Class vs Interface?

**Abstract Class:** Has abstract + non-abstract methods, constructors, static members. No multiple inheritance.

**Interface:** Only abstract methods. Supports multiple inheritance. All members public by default. No constructors.

**When Abstract?** Common functionality exists among classes.

**When Interface?** No common functionality. Useful for multiple inheritance, loose coupling, dependency injection.

**Can you implement only some methods?**

Yes make unimplemented ones abstract.

## Q: Overloading vs Overriding vs Hiding?

**Overloading:** Same name, different signatures (compile-time).

**Overriding:** Child class provides specific implementation of parent method (runtime).

**Method Hiding:** Uses 'new' keyword to hide base class method. Protects class member definitions.

# C# CLASS TYPES

## Abstract

Cannot instantiate, base for others

```
abstract class Shape { }
```

## Static

No instances, static members only

```
static class Helper { }
```

## Sealed

Cannot be inherited

```
sealed class Final { }
```

## Partial

Split across multiple files

```
partial class MyClass { }
```

## Singleton

Only one instance allowed

```
private static instance;
```

## Private (ctor)

Cannot create objects externally

```
private MyClass() { }
```

*Singleton uses private constructor. Used in .NET DI containers!*

↻ Repost

.NET Developer

Follow for more →

# C# CLASS TYPES CODE EXAMPLES

### Static Class

```
static class Helper
{
  public static int Add(int a, int b)
  { return a + b; }
}

int sum = Helper.Add(3, 5);
// No object needed, call directly
```

### Partial Class

```
// File1.cs
partial class Student
{ public string Name; }

// File2.cs
partial class Student
{ public int Age; }
// Same class, split in 2 files
```

### Struct (Value Type)

```
struct Point
{
  public int X, Y;
  public Point(int x, int y)
  { X = x; Y = y; }
}

Point p = new Point(10, 20);
```

### Record (Immutable Data)

```
record Person(
  string Name, int Age);

var p = new Person("Ravi", 25);
// Auto: Equals, ToString, GetHashCode

var p2 = p with { Age = 30 };
// Creates copy with changed Age
```

### Tuple (Multiple Returns)

```
(string name, int age) GetInfo()
{
  return ("Ravi", 25);
}

var info = GetInfo();
Console.Write(info.name);
```

### Anonymous Object

```
var emp = new
{
  Name = "Suresh",
  Dept = "IT"
};

// Quick object, no class needed
Console.Write(emp.Name);
```

### Extension Methods (add methods to existing class without changing it)

```
static class StringHelper
{
  public static bool IsUpper(this string s)
  { return s == s.ToUpper(); }
}

bool result = "HELLO".IsUpper(); // true
```

# C# CLASS TYPES DETAILED ANSWERS

## Abstract Class

Cannot be instantiated; acts as base class. Can have constructors (used by derived classes). Cannot be private or sealed.
**Purpose:** Place common functionality to reduce code duplication.

## Static Class

Cannot be instantiated. Contains only static members. Sealed by default no inheritance possible.
**Purpose:** Provide blueprints; compiler guarantees no instances.

## Partial Class

Split a class into multiple files. All parts combined at compilation.
**Advantage:** Multiple devs can work on same class simultaneously.

## Sealed Class

Cannot be inherited but CAN be instantiated.
**vs Private Class:** Cannot be inherited AND cannot be instantiated.

## Singleton Class

Can have only one object (instance). Implemented using private constructor.
**Uses:** Can be cloned, disposed; supports inheritance.

## Struct vs Class

**Struct:** Value type, stack memory, no inheritance.
**Class:** Reference type, heap memory, supports inheritance.

## Arrays & Jagged Arrays

Arrays: Same data type collection. Jagged: Array of arrays with different element sizes.
**Can store different types?** Yes, via object array or ArrayList.
**ArrayList vs Array:** ArrayList is slower (boxing/unboxing).

## Enum, String & Mutable

**Enum:** Value type with unique named constants.
**Immutable (string):** Cannot modify after creation. Thread safe.
**Mutable (StringBuilder):** Modifiable in place, no new memory.

## Q: Object, Properties, Indexers, Reflection?

**Object:** Real-world entity. Created via new operator, reference, or array.
**Properties:** Encapsulate getter/setter. **Indexers:** Array-like access using 'this' keyword.
**Reflection:** Describe metadata of types, methods, fields via System.Reflection.

# CONSTRUCTORS & DESTRUCTORS

### Default

No parameters, auto-created if none defined

```
MyClass() { }
```

### Parameterized

Accepts parameters to initialize fields

```
MyClass(int x) { }
```

### Copy

Creates new object from existing one

```
MyClass(MyClass obj) { }
```

### Static

Runs once, initializes static data

```
static MyClass() { }
```

### Private

Prevents external instantiation

```
private MyClass() { }
```

*Destructor: ~MyClass() { } called by GC. Only one per class!*

↻ Repost

Follow for more →

.NET Developer

# CONSTRUCTORS & DESTRUCTORS CODE

### Default Constructor

```
class Car
{
  public string Color;
  public Car()
  { Color = "Red"; }
}
// Called when: new Car()
```

### Parameterized Constructor

```
class Car
{
  public string Color;
  public Car(string c)
  { Color = c; }
}
// new Car("Blue")
```

### Copy Constructor

```
class Car
{
  public string Color;
  public Car(Car other)
  { Color = other.Color; }
}

var c1 = new Car { Color="Red" };
var c2 = new Car(c1); // Copy
```

### Static Constructor

```
class Config
{
  public static string AppName;

  static Config()
  { AppName = "MyApp"; }
}
// Runs once, before first use
// No access modifier, no params
```

### Private Constructor (Singleton)

```
class DB
{
  static DB _inst;
  private DB() { }
  public static DB Get()
  {
    if(_inst==null) _inst=new DB();
    return _inst;
  }
} // Only one object ever
```

### Destructor and IDisposable

```
class FileHandler : IDisposable
{
  ~FileHandler()
  { // GC calls this }

  public void Dispose()
  { // You call this }
}

using var fh = new FileHandler();
// auto-calls Dispose()
```

### Constructor Chaining (this and base)

```
class Person { public Person(string name) { } }
class Student : Person
{ public Student() : this("Default") { }
  public Student(string n) : base(n) { } }
```

# CONSTRUCTORS & DESTRUCTORS ANSWERS

## Constructor Types

**Default:** No parameters.
**Parameterized:** Takes parameters.
**Copy:** Takes class type param; initializes new object with existing values.
**Private:** Class cannot be inherited. Used for Singleton pattern.

## Static Constructor

Initializes static data. Invoked once regardless of how many instances are created.
**Rules:**
1. No access modifiers allowed
2. Must be parameterless
3. Cannot access non-static members
4. Cannot be inherited or overloaded

## Q: How many constructors in a class?

You can define any number of constructors, but each must have a different signature (different parameter types/count).
**Parent-Child relationship:** Parent constructor always fires first.

## Destructor

Destroys instances when no longer needed. A class can have only ONE destructor.
Called automatically by garbage collector.

## Method Types

**Virtual:** Has implementation, can be overridden.
**Abstract:** Definition only, forces override.
**Recursive:** Calls itself (e.g., file search in subfolders).

## Q: Object Initializers & Members?

**Object Initializers:** Syntax to quickly initialize an object without calling a constructor.
**Members:** Fields and methods inside classes. Properties encapsulate getter/setter for field access.

## Q: Serialization & Deserialization?

**Serialization:** Converting object into byte stream to save to memory, file, or database.
**Deserialization:** The reverse process byte stream back to object.

# ESSENTIAL C# KEYWORDS CODE

### this keyword

```
class Car
{ string color;
  public Car(string color)
  { this.color = color; }
} // this = current object
```

### base keyword

```
class Animal
{ public virtual void Sound()
  { Console.Write("..."); } }
class Dog : Animal
{ public override void Sound()
  { base.Sound(); } }
```

### is and as keywords

```
object obj = "Hello";

// is: checks type
if (obj is string) { }

// as: safe cast (null if fails)
string s = obj as string;
```

### typeof, nameof, sizeof

```
// typeof: gets Type at compile time
Type t = typeof(int);

// nameof: gets name as string
string n = nameof(Console);
// n = "Console"

int size = sizeof(int); // 4
```

### ref and out parameters

```
void Double(ref int x)
{ x = x * 2; }

void GetName(out string n)
{ n = "Ravi"; }

// ref: must init before passing
// out: assigned inside method
```

### params keyword

```
int Sum(params int[] nums)
{
  int total = 0;
  foreach(var n in nums)
    total += n;
  return total;
}
Sum(1,2,3,4); // 10
```

### using keyword

```
// Auto-disposes resource
using (var file = File.Open(
  "data.txt", FileMode.Open))
{
  // file auto-closed here
}
```

### lock (thread safety)

```
object _lock = new object();

lock(_lock)
{
  count++;
} // Only 1 thread at a time
```

### yield keyword (lazy return)

```
IEnumerable<int> GetNums() { yield return 1; yield return 2; yield return 3; }
foreach(var n in GetNums()) Console.Write(n); // 1 2 3
```

# ESSENTIAL C# KEYWORDS ANSWERS

## Core Keywords

**this** Current instance of the class.
**base** Access base class members from derived.
**static** Use without creating instance; saves memory.
**virtual** Allows derived class to change implementation.
**override** Derived class provides its own implementation.
**sealed** Restricts inheritance of the class.
**params** Receive multiple number of parameters.

## Parameter & Exception Keywords

**ref** Pass by reference (bidirectional).
**out** Pass by reference (unidirectional). Returns multiple values.
**try** Place code that may raise exception.
**catch** Handle the exception.
**finally** Executes whether or not exception thrown.
**throw** Manually throw exception during execution.
**using** Calls Dispose to free resources; imports namespaces.

## var vs dynamic

**var:** Type decided at compile time. Type safe. Initialization required. Supports IntelliSense.
**dynamic:** Type decided at runtime. Not type safe. No initialization needed. No IntelliSense.

## ref vs out

**ref:** Bidirectional data passing. Parameter initialization required.
**out:** Unidirectional (getting data). No initialization needed.

## throw vs throw ex

**throw:** Provides where exception was thrown + exception details.
**throw ex:** Only provides where exception was thrown.

## is vs as

**is:** Returns true/false if object can be cast to a type.
**as:** Attempts cast, returns null if it fails.

## Exception Handling

A process to handle runtime errors maintaining normal application flow. Use try-catch block. The advantage of immutability is thread safety.

# C# COLLECTIONS

## GENERIC (Type-Safe)

**List<T>**
Dynamic array, indexed

**Dictionary<K,V>**
Key-value pairs, fast lookup

**HashSet<T>**
Unique elements only

**Queue<T>**
FIFO (First In First Out)

**Stack<T>**
LIFO (Last In First Out)

**SortedList<K,V>**
Auto-sorted by key

## NON-GENERIC (Object)

**ArrayList**
Like List but stores objects

**Hashtable**
Like Dictionary, not typed

**Stack**
Non-generic LIFO

**Queue**
Non-generic FIFO

**SortedList**
Auto-sorted, not typed

**BitArray**
Compact boolean array

KEY INTERFACES: IEnumerable | ICollection | IList | IQueryable | IDictionary

↻ Repost

.NET Developer

Follow for more →

# C# COLLECTIONS CODE EXAMPLES

### List<T> (dynamic array)

```
List<string> names = new();
names.Add("Ravi");
names.Add("Suresh");
names.Remove("Ravi");

foreach(var n in names)
  Console.Write(n);
```

### Dictionary<K,V> (key-value)

```
var scores = new Dictionary
  <string, int>();
scores["Ravi"] = 90;
scores["Suresh"] = 85;

int s = scores["Ravi"];
// s = 90
```

### Stack<T> (LIFO)

```
var stack = new Stack<int>();
stack.Push(10);
stack.Push(20);
stack.Push(30);

int top = stack.Pop(); // 30
int peek = stack.Peek(); // 20
```

### Queue<T> (FIFO)

```
var q = new Queue<string>();
q.Enqueue("A");
q.Enqueue("B");
q.Enqueue("C");

string first = q.Dequeue();
// first = "A"
```

### HashSet<T> (unique items)

```
var set = new HashSet<int>();
set.Add(1); set.Add(2); set.Add(1);
// set has {1, 2} only

bool has = set.Contains(2);
// true
```

### LinkedList<T>

```
var ll = new LinkedList<int>();
ll.AddLast(10);
ll.AddFirst(5);
ll.AddLast(20);

// Result: 5, 10, 20
// Fast insert/remove at ends
```

### LINQ (query any collection)

```
List<int> nums = new() { 1,2,3,4,5,6,7,8 };

var evens = nums.Where(n => n % 2 == 0);  // 2,4,6,8
var first = nums.First(n => n > 3);       // 4
int sum   = nums.Sum();                    // 36
```

### Array vs List (key difference)

```
int[] arr = new int[3];  // Fixed size = 3, cannot grow
List<int> list = new();   // Dynamic, grows as needed
list.Add(1); list.Add(2); // Keep adding, no limit
```

# C# COLLECTIONS DETAILED ANSWERS

## Generic Collections

Hold elements of same data type:

- **List<T>** Specified type elements.
- **Dictionary<TK,TV>** Key-value pairs.
- **SortedList<TK,TV>** Ascending order KV pairs.
- **Queue<T>** FIFO (Enqueue/Dequeue).
- **Stack<T>** LIFO (Push/Pop/Peek).
- **HashSet<T>** Non-duplicate elements.

## Non-Generic Collections

Hold elements of different data types:

- **ArrayList** Stores any object type.
- **SortedList** Ascending order KV pairs.
- **Stack** LIFO style.
- **Queue** FIFO style.
- **Hashtable** KV pairs via hash comparison.
- **BitArray** Compact bit values as booleans.

## Collection Interfaces Explained

**IEnumerable:** Supports enumeration (foreach).
**ICollection:** Enumerable + element count.
**IList:** Add, remove, retrieve by index. Most commonly used.
**IQueryable:** Supports LINQ queries.
**IDictionary:** Enumerate key-value pairs.

## Dictionary vs Hashtable

**Dictionary:** Generic. Same type KV pairs. Faster retrieval. Maintains order.
**Hashtable:** Non-generic. Mixed type KV pairs. Slower retrieval. No order guarantee.
**Can you store different lists in one generic list?** Yes, via List<List<object>>.

## Array vs ArrayList

**Array:** Fixed size. Strongly typed. Better performance.
**ArrayList:** Dynamic size. Not strongly typed. Slower (boxing/unboxing).

# DELEGATES, EVENTS & ASYNC

## DELEGATES = Type-Safe Function Pointers

| | | |
|---|---|---|
| Delegate | Custom function pointer | `delegate void MyDel(int x);` |
| Func<T> | Has return value (0-16 params) | `Func<int, string> convert;` |
| Action<T> | No return value (void) | `Action<string> log;` |
| Predicate<T> | Returns bool (filter/find) | `Predicate<int> isEven;` |

## EVENTS

Publisher-Subscriber pattern

1. Create a delegate
2. Define event using delegate
3. Raise the event

## ASYNC / AWAIT

Run tasks in parallel

```
async marks the method
await pauses until done
Task wraps the operation
```

## EXCEPTION HANDLING

```
try { risky code }  catch { handle }  finally { always runs }  throw { manual }
```

↻ Repost

.NET Developer

Follow for more →

# DELEGATES, EVENTS & ASYNC CODE

### Delegate

```
delegate int MathOp(int a, int b);

int Add(int a, int b)
{ return a + b; }

MathOp op = Add;
int result = op(3, 5);
// result = 8
```

### Lambda Expression

```
// Short way to write a method

Func<int,int,int> add =
  (a, b) => a + b;

Action<string> greet =
  name => Console.Write(name);

greet("Ravi"); // Ravi
```

### Event (notify subscribers)

```
class Button
{
  public event Action OnClick;

  public void Click()
  { OnClick?.Invoke(); }
}

var btn = new Button();
btn.OnClick += () =>
  Console.Write("Clicked!");
btn.Click();
```

### Async / Await

```
async Task<string> GetDataAsync()
{
  await Task.Delay(1000);
  return "Done";
}

// Call it:
string data =
  await GetDataAsync();
// Waits 1 sec, then "Done"
// App stays responsive!
```

### Func, Action, Predicate

```
// Func: has return value
Func<int,int> double_it = x => x*2;

// Action: no return (void)
Action<string> log = s =>
  Console.Write(s);

// Predicate: returns bool
Predicate<int> isEven = n => n%2==0;
```

### Try-Catch-Finally

```
try
{ int x = 10 / 0; }
catch(DivideByZeroException e)
{ Console.Write(e.Message); }
finally
{ Console.Write("Always runs"); }

// try = risky code
// catch = handle error
```

### Parallel Async (run multiple tasks at once)

```
var t1 = GetUsersAsync();  // starts
var t2 = GetOrdersAsync(); // starts
await Task.WhenAll(t1, t2);  // waits for both, runs in parallel!
```

# DELEGATES, EVENTS & ASYNC ANSWERS

## Q: What are Delegates?

A type-safe function pointer. Acts as a representative, establishing communication between publisher and subscriber via callbacks.

**Multicast Delegate:** Allows multiple function pointers in a single delegate.

## Q: Func vs Action vs Predicate?

**`Func<T>:`** Generic delegate with 0+ input params and ONE output parameter.

**`Action<T>:`** Generic delegate with input params (up to 16) and NO return value.

**`Predicate<T>:`** Delegate with ONE input param and bool return type. Used for filtering.

**Predicate return type:** `bool`

## Q: How do Events work?

Events establish communication between objects for loosely coupled applications. Helps extend applications.

**Three steps for events:**

1. Create a delegate , 2. Define event based on delegate , 3. Raise the event

## Q: Async & Await?

Asynchronous programming enables running tasks in parallel.

**Task:** Object encapsulating the state of an async operation.

**async/await:** Code markers indicating where control resumes after task completion.

**When to use:** Long-running tasks (file I/O). Advantage: Overcomes GUI blocking.

# TYPE CONVERSION & STRINGS

## TYPE CONVERSION

### Implicit (Auto)

int to long, float to double
Compiler handles, no data loss

### Explicit (Cast)

(int)myDouble
Convert.ToInt32(), data loss!

## String (Immutable)

Cannot modify after creation
New object each change
Thread-safe
*Use for: few modifications*

## StringBuilder (Mutable)

Modifies in-place
No new memory allocation
Better performance in loops
*Use for: many modifications*

`ToString() throws NullReferenceException | Convert.ToString() handles null safely`

Array is faster than ArrayList (no boxing overhead)

↻ Repost

.NET Developer

Follow for more →

# TYPE CONVERSION & STRINGS CODE

### Parse and TryParse

```
// Parse: crashes if invalid
int n = int.Parse("42");

// TryParse: safe, returns bool
bool ok = int.TryParse(
  "abc", out int val);
// ok = false, val = 0
```

### Convert Class

```
int a = Convert.ToInt32("100");
double d = Convert.ToDouble("3.14");
string s = Convert.ToString(42);
bool b = Convert.ToBoolean(1);

// Works with many types
// Handles null better than Parse
```

### String Interpolation

```
string name = "Ravi";
int age = 25;

// Old way:
"Name: " + name + ", Age: " + age

// New way (cleaner):
$"Name: {name}, Age: {age}"
```

### Common String Methods

```
string s = " Hello World ";
s.Trim()      // "Hello World"
s.ToUpper()   // "HELLO WORLD"
s.ToLower()   // "hello world"
s.Contains("World") // true
s.Replace("World","C#")
```

### Verbatim and Raw Strings

```
// Verbatim: no escape needed
string path = @"C:\Users\file";

// Multiline string
string msg = @"Line 1
Line 2
Line 3";
```

### Pattern Matching (switch)

```
object val = 42;

string result = val switch
{
  int n when n > 0 => "Positive",
  string s => s,
  _ => "Unknown"};
```

### Null Safety Operators

```
string? name = null;

// ?? = default if null
string display = name ?? "Guest";

// ?. = skip if null
int? len = name?.Length;
```

### StringBuilder (fast concat)

```
var sb = new StringBuilder();
sb.Append("Hello");
sb.Append(" World");
sb.Replace("World","C#");
string result = sb.ToString();
```

### Split and Join

```
string[] parts = "A,B,C".Split(',');  // ["A","B","C"]
string joined = string.Join("-", parts); // "A-B-C"
```

# TYPE CONVERSION & STRINGS ANSWERS

## Q: What is Type Conversion?

Converting one data type into another.

**Implicit:** Done by C# compiler in a type-safe manner (e.g., int to long).

**Explicit:** Done by user using pre-defined functions (e.g., (int)doubleValue).

## Q: ToString() vs Convert.ToString()?

Both convert a value to a string.

**ToString():** Does NOT handle null throws NullReferenceException.

**Convert.ToString():** HANDLES nulls safely.

## String (Immutable)

Cannot modify after creation.

Belongs to System namespace.

Uses concatenation (+) to combine.

**Performance degrades with many manipulations.**

**Advantage:** Thread safe.

## StringBuilder (Mutable)

CAN modify after creation.

Belongs to System.Text namespace.

Uses Append() method to combine.

**Better performance for many manipulations.**

**Updates string in place,** no new memory.

## Static Class vs Singleton Class

**Static:** Cannot create instance. Cannot clone. Cannot dispose. No inheritance.

**Singleton:** Can create ONE instance and reuse it. Can clone. Can dispose. Supports inheritance.

## GC.Collect, Dispose & Finalize

**GC.Collect:** Auto-called for managed resources. **Dispose:** Explicit, for unmanaged (IDisposable). **Finalize:** Implicit, for DB connections.

# MEMORY MANAGEMENT & ADVANCED

## GARBAGE COLLECTION & CLEANUP

| | | |
|---|---|---|
| `GC.Collect()` | Auto-called for managed resources | Managed |
| `Dispose()` | Call explicitly via IDisposable | Unmanaged |
| `Finalize()` | Called when object eligible for GC | Unmanaged |
| `using { }` | Auto-calls Dispose() on scope end | Best Practice |

## SERIALIZATION

Object to byte stream
Deserialization = reverse
Used for: DB, files, APIs

## REFLECTION

Inspect types at runtime
System.Reflection namespace
Used for: DI, ORMs, testing

## BOXING & UNBOXING

```
Boxing: int to object
```

```
Unboxing: object to int
```

*Both hurt performance. Use generics to avoid!*

↻ Repost

.NET Developer

Follow for more →

# MEMORY MANAGEMENT & ADVANCED CODE

### Stack vs Heap

```
// Stack: value types (fast, auto)
int x = 10; // on Stack

// Heap: reference types (GC manages)
string s = "Hello"; // on Heap
int[] arr = {1,2}; // on Heap
// 's' and 'arr' refs on Stack
```

### IDisposable Pattern

```
class DbConn : IDisposable
{
  public void Dispose()
  { // close connection }
}

using var db = new DbConn();
// auto-disposed when scope ends
```

### Garbage Collection

```
var obj = new object();
obj = null; // eligible for GC

// Force GC (rarely needed)
GC.Collect();
GC.WaitForPendingFinalizers();
// Let GC handle it usually!
```

### WeakReference

```
var obj = new object();
var weak = new WeakReference(obj);

obj = null; // only weak ref left
GC.Collect();

if (weak.IsAlive)
  Console.Write("Still here");
```

### Reflection (inspect at runtime)

```
Type t = typeof(String);

// Get all methods
var methods = t.GetMethods();
foreach(var m in methods)
  Console.Write(m.Name);

// Get all properties
var props = t.GetProperties();
```

### Custom Attributes

```
class InfoAttribute : Attribute
{ public string Author; }

[Info(Author = "Ravi")]
class MyClass { }

// Read it via Reflection:
var attr = typeof(MyClass)
  .GetCustomAttribute<InfoAttribute>();
```

### Dependency Injection

```
interface ILogger
{ void Log(string msg); }

class App
{ private ILogger _log;
  public App(ILogger log)
  { _log = log; } // injected!
}
```

### Indexer (access like array)

```
class Team
{ string[] members = new string[5];
  public string this[int i]
  { get => members[i];
    set => members[i] = value; } }

var t = new Team();
t[0] = "Ravi"; // indexer!
```

### Span<T> (zero-copy slicing)

```
int[] data = {1,2,3,4,5}; Span<int> slice = data.AsSpan(1,3); // {2,3,4}
```

# MEMORY MANAGEMENT & ADVANCED ANSWERS

## Q: Stack vs Heap Memory?

**Stack:** Allocated at compile time. Stores value types. Immediately removed when out of scope. Fast access.
**Heap:** Allocated at runtime. Stores reference types. Cleaned by Garbage Collector (CLR). Slower access.
**Key Point:** Value types on stack are deterministic; heap objects need GC for cleanup.

## Generics Advantages

1. **Reusability** Write once, use with any type.
2. **Type Safety** Compile-time type checking.
3. **Performance** No boxing/unboxing overhead.

## Fields & Members

**Field:** Variable declared at class level.
**Method:** Group of statements performing a task.
**Members:** Fields + methods inside classes.

## Q: OOP Four Pillars Summary

**Encapsulation:** Wrap code + data; private vars + public properties.
**Inheritance:** Acquire parent properties/behaviors; enables code reuse.
**Polymorphism:** Multiple forms; overloading (compile) + overriding (runtime).
**Abstraction:** Hide details, show essentials; abstract classes/interfaces.

## Serialization

Object byte stream (to save to memory/file/DB).
**Deserialization:** Byte stream object (reverse process).

## Reflection & Indexers

**Reflection:** Describe metadata of types/methods/fields at runtime.
**Indexers:** Array-like access for objects using 'this' keyword.

## Exception Handling

Process to handle runtime errors maintaining normal application flow. Use try/catch/finally. Throw keyword for manual exceptions. Using keyword for IDisposable cleanup.

That's the complete C# cheat sheet!

# SAVE + SHARE

with someone learning C#

**Save**

Bookmark for interviews

**Like**

If this was helpful

**Share**

Help a fellow dev

**Follow**

For more .NET content


## Jagadish Rapuru

.NET & AI Engineer | 5 Years Experience | Azure Certified

.NET Developer

↻ Repost

Follow for more →